L2 ANDROID MASWE-0116 TEST

MASTG-TEST-0223: Stack Canaries Not Enabled

Overview

This test case checks if the native libraries of the app are compiled without common binary protection mechanisms (*Binary Protection Mechanisms*) such as stack smashing protection, a mitigation technique against buffer overflow attacks.

- NDK libraries should have stack canaries enabled since the compiler does it by default ↗.
- Other custom C/C++ libraries might not have stack canaries enabled because they lack the necessary compiler flags (`-fstack-protector-strong`, or `-fstack-protector-all`) or the canaries were optimized out by the compiler. See the Evaluation section for more details.

Steps

1. Extract the app contents (*Exploring the App Package*).
2. Run *Obtaining Compiler-Provided Security Features* on each shared library and grep for "canary" or the corresponding keyword used by the selected tool.

Observation

The output should show if stack canaries are enabled or disabled.

Evaluation

The test case fails if stack canaries are disabled.

Developers need to ensure that the flags `-fstack-protector-strong`, or `-fstack-protector-all` are set in the compiler flags for all native libraries. This is especially important for custom C/C++ libraries that are not part of the NDK.

When evaluating this please note that there are potential **expected false positives** for which the test case should be considered as passed. To be certain for these cases, they require manual review of the original source code and the compilation flags used.

The following examples cover some of the false positive cases that might be encountered:

Use of Memory Safe Languages

The Flutter framework does not use stack canaries because of the way Dart mitigates buffer overflows ↗.

Compiler Optimizations

Sometimes, due to the size of the library and the optimizations applied by the compiler, it might be possible that the library was originally compiled with stack canaries but they were optimized out. For example, this is the case for some react native apps ↗. They are built with `-fstack-protector-strong` but when attempting to search for `stack_chk_fail` inside the `.so` files, it is not found.

- **Empty .so files**: Some .so files such as `libruntimeexecutor.so` or `libreact_render_debug.so` are effectively empty in release and therefore contain no symbols. Even if you were to attempt to build with `-fstack-protector-all`, you still won't be able to see the `stack_chk_fail` string as there are no method calls there.
- **Lack of stack buffer calls**: Other files such as `libreact_utils.so`, `libreact_config.so`, and `libreact_debug.so` are not empty and contain method calls, but those methods don't contain stack buffer calls, so there are no `stack_chk_fail` strings inside them.

The React Native developers in this case declare that they won't be adding `-fstack-protector-all` as, in their case, they consider that doing so will add a performance hit for no effective security gain.